



CONCORRÊNCIA E MULTITHREADING

AULA 02 - EXECUTORS E THREAD POOLS



AGENDA

- ❑ Executors e Thread Pools: Gerenciamento Moderno de Threads
- ❑ Abstraindo a execução de tarefas do gerenciamento de threads.
- ❑ Do controle manual à automação e eficiência.



EXECUTOR FRAMEWORK: COMPONENTES FUNDAMENTAIS

- ❑ Interface Executor: O contrato mais simples: void execute(Runnable command). Desacopla a submissão da execução.
- ❑ Interface ExecutorService: Estende Executor. Adiciona gerenciamento de ciclo de vida (shutdown, awaitTermination), submissão de Callable, e obtenção de resultados via Future.
- ❑ Classe Executors: Classe factory para criar instâncias de ExecutorService com configurações pré-definidas e otimizadas.



TIPOS DE POOLS DE THREADS (PLATFORM THREADS)

- ❑ `newFixedThreadPool(int n)`: Mantém um número fixo de threads. Ideal para cargas de trabalho com uso intensivo de CPU, onde o número de threads é dimensionado de acordo com os núcleos do processador.
- ❑ `newCachedThreadPool()`: Cria threads conforme a necessidade e as reutiliza. Adequado para um grande número de tarefas curtas e assíncronas.
- ❑ `newSingleThreadExecutor()`: Garante que as tarefas sejam executadas sequencialmente em uma única thread. Útil para garantir ordem e evitar problemas de concorrência em tarefas de background.
- ❑ `newScheduledThreadPool(int corePoolSize)`: Permite agendar a execução de tarefas após um atraso ou periodicamente.



DIMENSIONANDO POOLS DE THREADS: CPU-BOUND VS. I/O-BOUND

- ❑ Tarefas CPU-Bound: O número de threads deve ser próximo ao número de núcleos de CPU. Um pool com NCPUs ou NCPUs +1 threads maximiza a utilização sem excesso de troca de contexto.
- ❑ Tarefas I/O-Bound: As threads passam muito tempo bloqueadas (esperando por rede, disco). Um número maior de threads é necessário para manter a CPU ocupada.
- ❑ Fórmula de Dimensionamento (Brian Goetz):
$$\text{Threads} = \text{NCPUs} * (1 + \text{Tempo de Espera} / \text{Tempo de Serviço})$$



A REVOLUÇÃO DO PROJETO LOOM: VIRTUAL THREADS

- ❑ Java 21+: Introdução das Virtual Threads (finalizadas). Threads leves gerenciadas pela JVM, não pelo SO.
- ❑ `Executors.newVirtualThreadPerTaskExecutor()`: Um novo tipo de `ExecutorService` que cria uma nova thread virtual para cada tarefa submetida.
- ❑ Benefícios: Baixíssimo custo de criação e bloqueio. Permite milhões de threads concorrentes, ideal para tarefas I/O-bound.



CICLO DE VIDA E DESLIGAMENTO (SHUTDOWN)

- ❑ `shutdown()`: Desligamento ordenado. Rejeita novas tarefas, mas permite que as tarefas já submetidas terminem.
- ❑ `shutdownNow()`: Desligamento abrupto. Tenta parar todas as tarefas em execução (via `Thread.interrupt()`) e retorna a lista de tarefas que não foram iniciadas.
- ❑ `awaitTermination(long timeout, TimeUnit unit)`: Bloqueia a execução até que todas as tarefas terminem após um shutdown, ou até que o tempo limite expire.
- ❑ Melhor Prática: Sempre desligue seus `ExecutorServices`, preferencialmente em um bloco `finally` ou usando `try-with-resources`.



ATÉ A PRÓXIMA!